

CPSC 533V: Learning to Move

(Reinforcement Learning)

Michiel van de Panne

Department of Computer Science
The University of British Columbia



Registration

- we're looking for a larger room
- 27 registered, 10+ looking to register
- lectures will be recorded until at least drop deadline, links posted on Piazza -> lectures

Lecture Plan Wed Sept 4, 2024

- what is special about movement?
- course administrivia
- what is RL?
 - what can it solve? how does it differ from other ML problems?
- why Deep RL, and some caveats
- icebreaker

Movement – what is so special?

- Q: Why do we have a brain?
- A: To produce adaptable and complex movements [Wolpert]
- Movement is the only* way we have of:
 - interacting with the world
 - communicating: speech, gestures, writing
- “motor control chauvinism”
 - chau·vin·ism [shoh-vuh-niz-uhm]
 - 2. biased devotion to any group, attitude, or cause
- “athletic intelligence”

Why do we have a brain?



- because computers were not yet invented
- to behave

- example: sea squirt

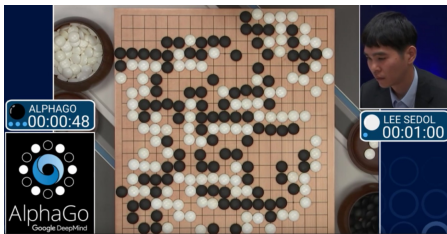


- larval stage: primitive brain & eye, swims around, attaches to a rock
- adult stage: sits. digests brain.

Example motions

- (videos)

What is RL?

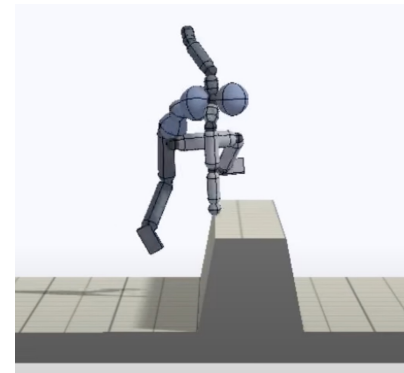


[AlphaGo]



Wayve's autonomous cars 'learning' to drive in London...
trafficttechnologytoday.com

[Wayve]



[DeepMimic]



[ANYmal, ETHZ]

The lecture materials are based in part on, or inspired by, slides/blogs/demos from:

- Rich Sutton, U Alberta [Sutton] <http://incompleteideas.net/book/the-book-2nd.html>
- Emma Brunskill, Stanford [Brunskill] <https://web.stanford.edu/class/cs234/index.html>
- David Silver, UCL [Silver] <http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html>
- Xue Bin Peng, SFU [Peng]
- Pieter Abbeel, UC Berkeley [Abbeel]
- Sergey Levine, Chelsea Finn, John Schulman, UC Berkeley [UC Berkeley]
- Mark Schmidt, UBC [Schmidt]
- Andrej Karpathy [Karpathy] <https://karpathy.github.io/2016/05/31/rl/>
- Lillian Weng [Weng] <https://lilianweng.github.io/lil-log/2018/04/08/policy-gradient-algorithms.html>

Learn to make good sequences of decisions

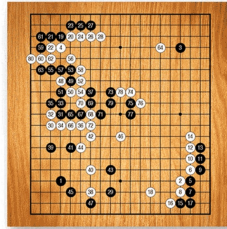
[Brunskill]

- Drive a car
- Defeat the world champion at Go
- Manage an investment portfolio
- Sequence a series of medical tests and interventions
- Control a power station or a chemical process to maximize revenue
- Make a humanoid robot walk
- Direct attention for a computer vision task
- Decide on the next medical investigation or treatment

Defining good decisions: rewards

- Fly stunt manoeuvres in a helicopter
 - +ve reward for following desired trajectory
 - -ve reward for crashing
 - Defeat the world champion at Backgammon
 - +/-ve reward for winning/losing a game
 - Manage an investment portfolio
 - +ve reward for each \$ in bank
 - Control a power station
 - +ve reward for producing power
 - -ve reward for exceeding safety thresholds
 - Make a humanoid robot walk
 - +ve reward for forward motion
 - -ve reward for falling over
 - Play many different Atari games better than humans
 - +/-ve reward for increasing/decreasing score
- [Abbeel]

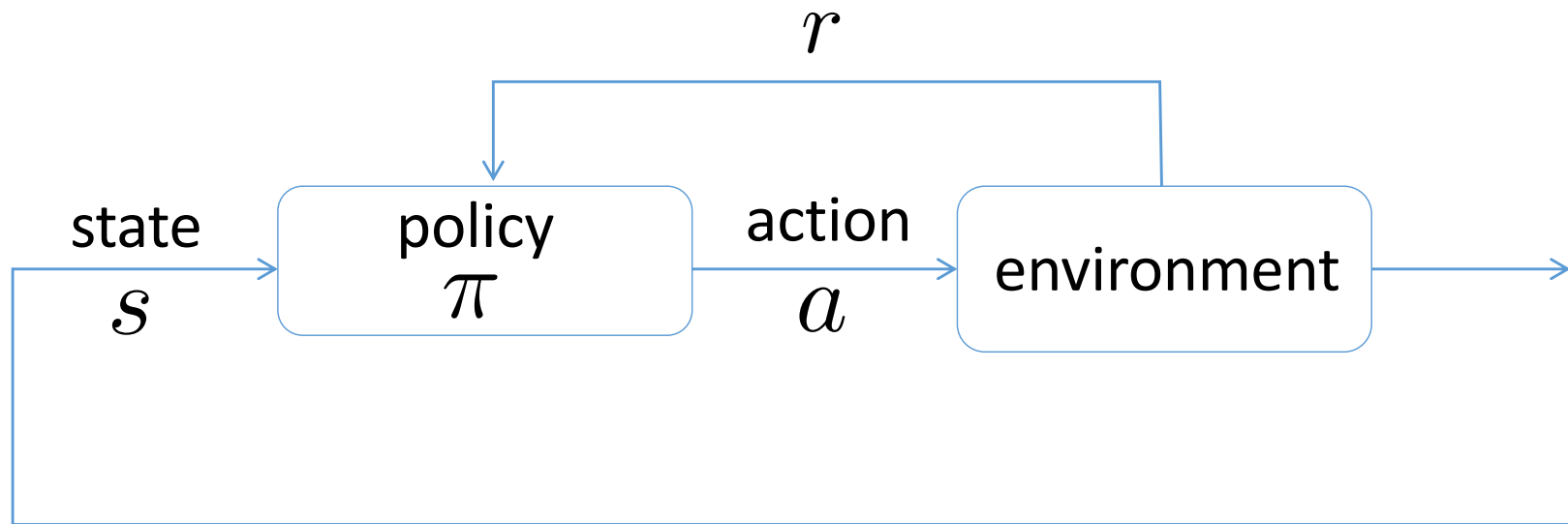
RL involves:



- optimization
 - find an optimal way to make sequential decisions
- delayed consequences
 - decisions now can impact things much later, e.g., climate change
 - challenge: temporal credit assignment is hard (what caused later high or low rewards?)
 - challenge: need to reason about long-term ramifications
- exploration
 - explore vs exploit tradeoff: try a new restaurant or go to one you already like?
 - current policy impacts future data collection: potential instabilities
- generalization
 - impossible to visit all states during learning, e.g. image input to a policy

~[Brunskill]

Reinforcement Learning



maximize sum of rewards: $G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

$$0 \leq \gamma \leq 1$$

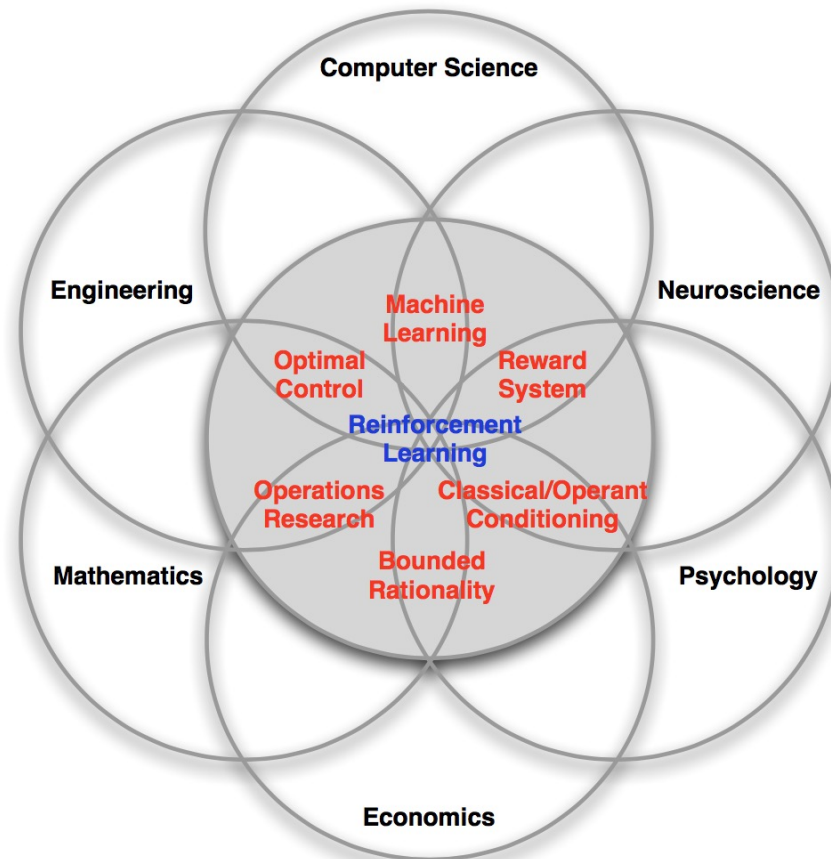
Discrete vs Continuous

- discrete states, discrete actions
 - e.g., Go, Chess, tic-tac-toe
 - “tabular” policies for small problems
- continuous states, discrete actions
 - e.g., Atari games, DOTA, cart-and-pole
- continuous states, continuous actions
 - robotics, process control, autonomous driving
- discrete states, continuous actions – less common?

RL (and ML) "Word Salad"

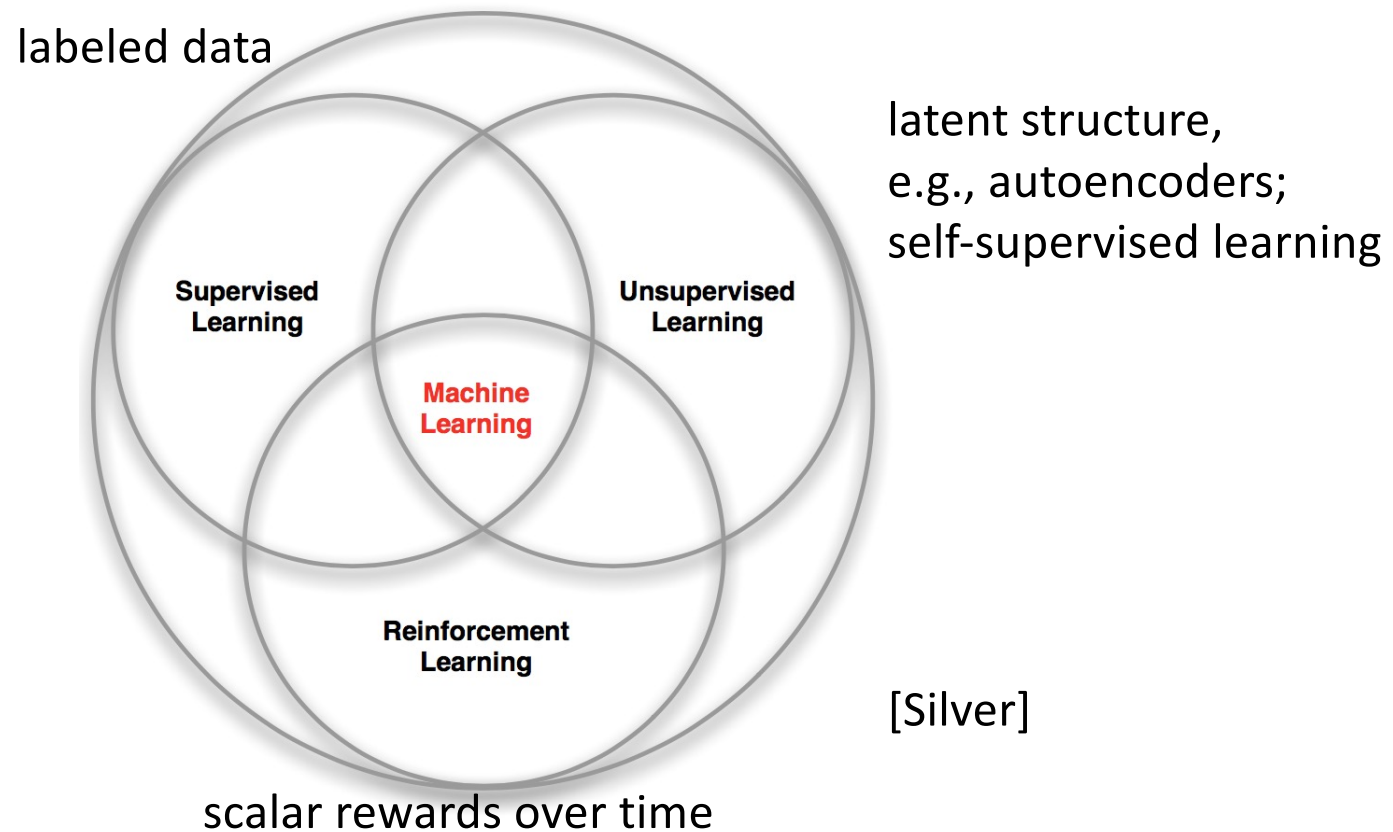
behavior cloning hindsight replay early termination
MDP LLMs
actor MPC dynamic programming
foundation model entropy bonus state-action value fn transfer learning
imitation learning inverse RL DDPM On policy vs Off policy
value fn critic options experience replay POMDP DAGGER Online vs Offline
DDPG decision transformer GAN bandits aleatoric vs epistemic uncertainty
bootstrapping TRPO, PPO policy gradient deadly triad

The Many Faces of Reinforcement Learning



[Silver]

Branches of Machine Learning

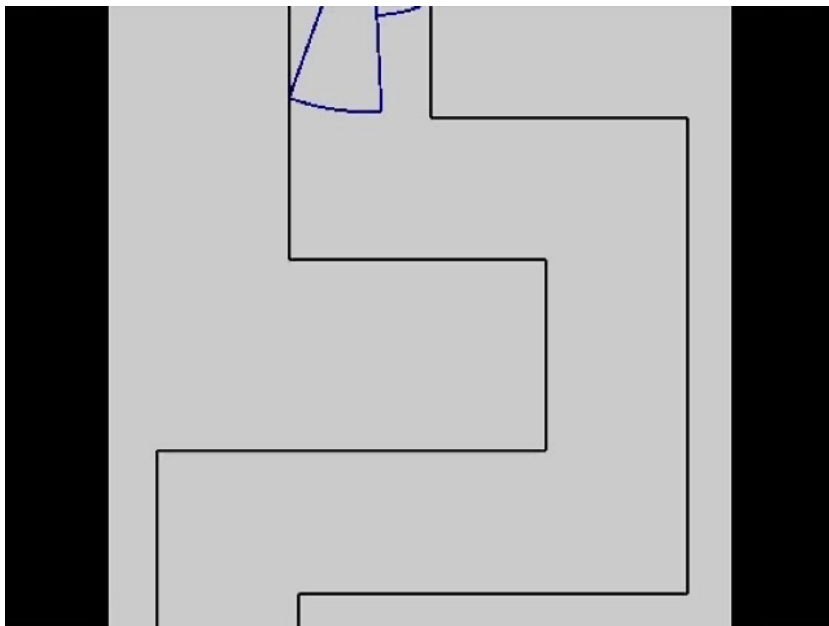


RL is the OG...

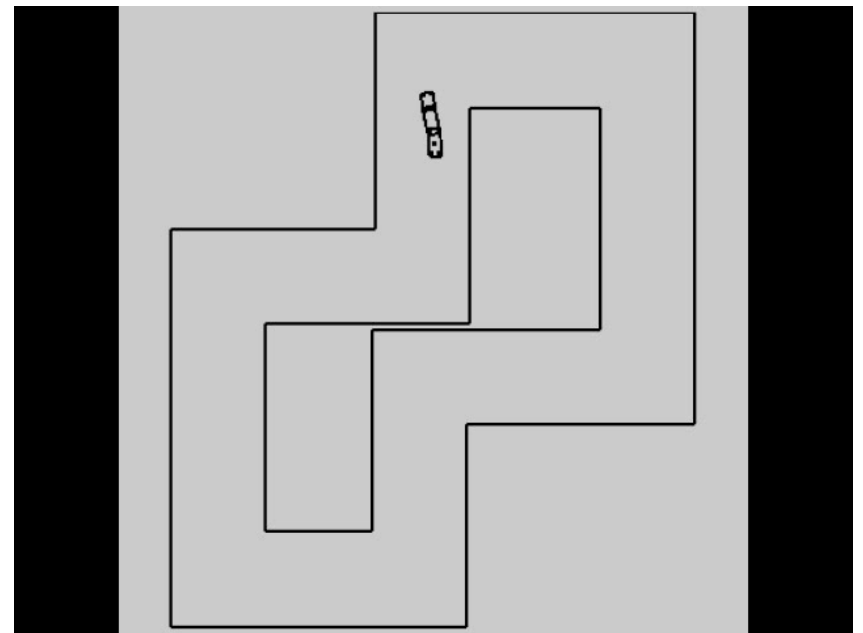
- “In the beginning, ML was RL” – Andrew Barto
 - combines SEARCH and MEMORY
 - search: trial-and-error (data collection)
 - memory: cache solutions, i.e., policy, value-function

Simple Example: Backing up a Truck

$\pi : \mathbb{R}^5 \rightarrow \mathbb{R}$ input: 4 distances + cab-angle
output: steering angle



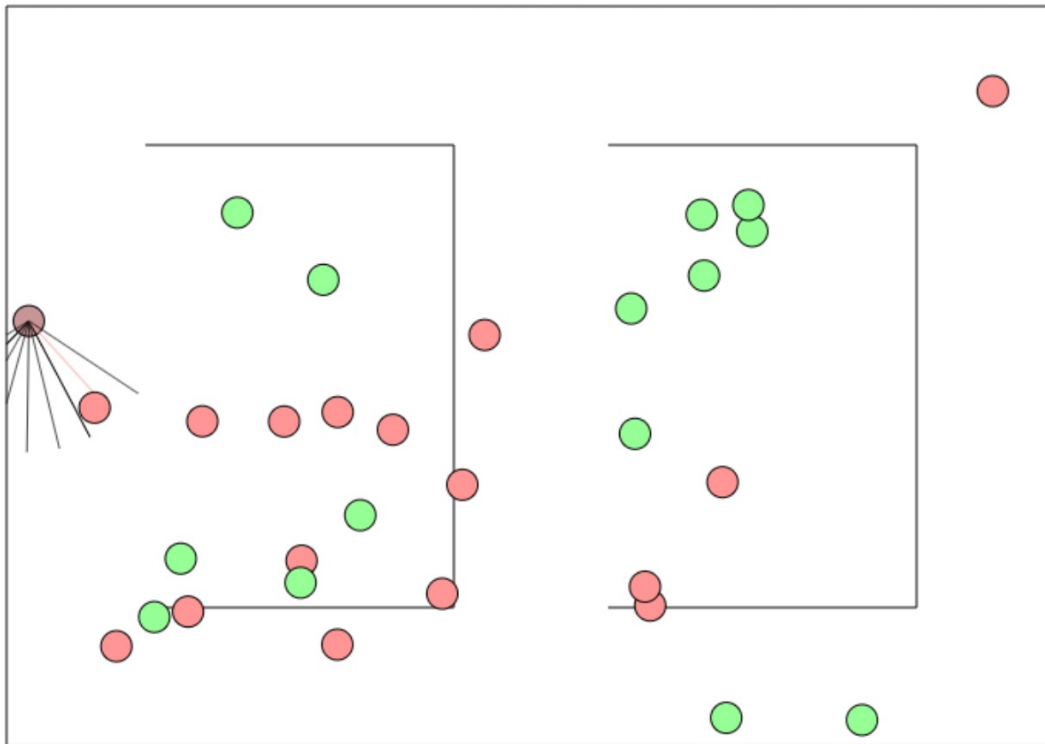
backing up a double trailer



[“Learning to Steer on Winding Tracks Using Semi-Parametric Control Policies”, ICRA 2005]

Simple Example

state: 9 sensors x 3 values x 2 timesteps
actions: move in 5 directions



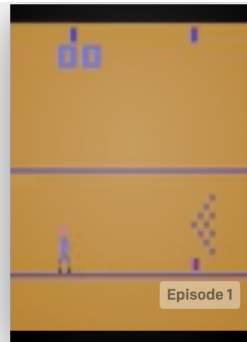
[<https://cs.stanford.edu/people/karpathy/convnetjs/demo/rldemo.html>]

OpenAI Gym

[<https://openai.com/research/openai-gym-beta>]



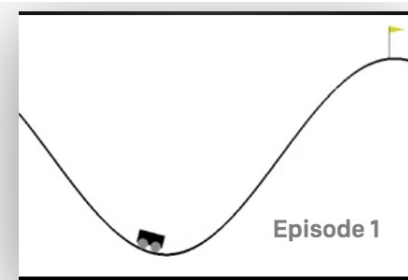
Berzerk-v0



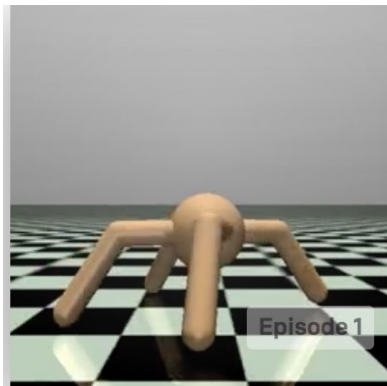
Bowling-ram-v0



CartPole-v1



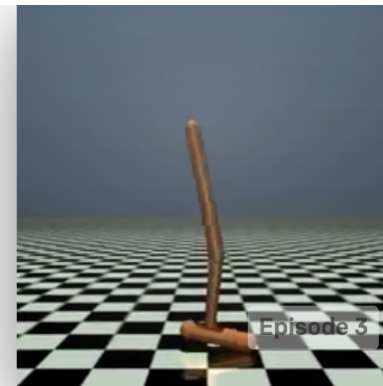
MountainCar-v0



Ant-v2



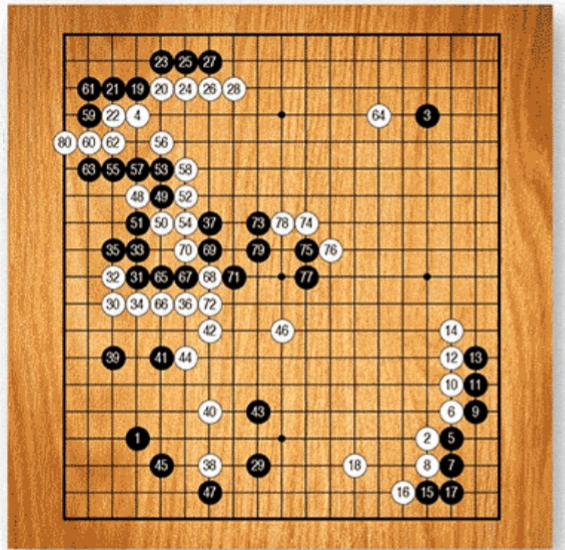
HalfCheetah-v2



Hopper-v2

Why Deep RL?

AlphaZero (DeepMind)

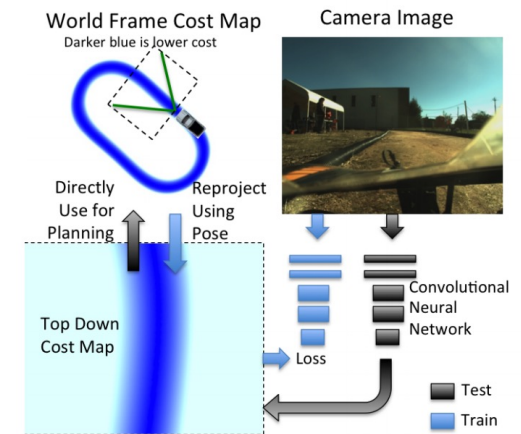


[DeepMind: Nature 2017]

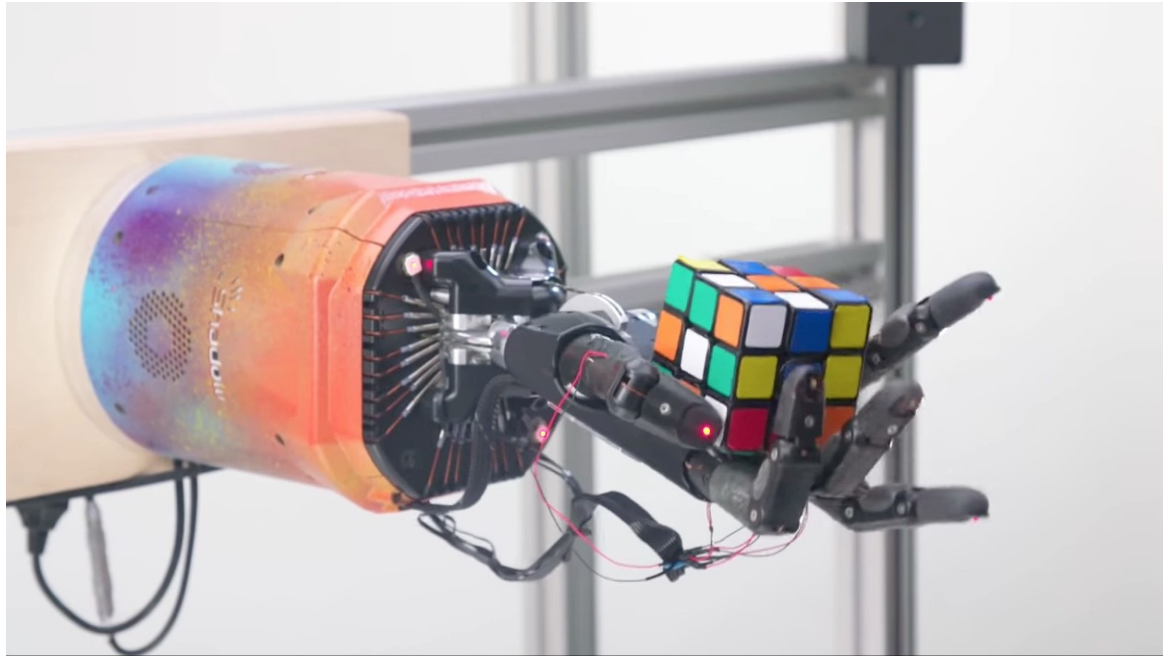
Aggressive autonomous driving of a scale vehicle

(Georgia Tech)

[Georgia Tech: CoRL 2017]



Rubik's cube manipulation (OpenAI)



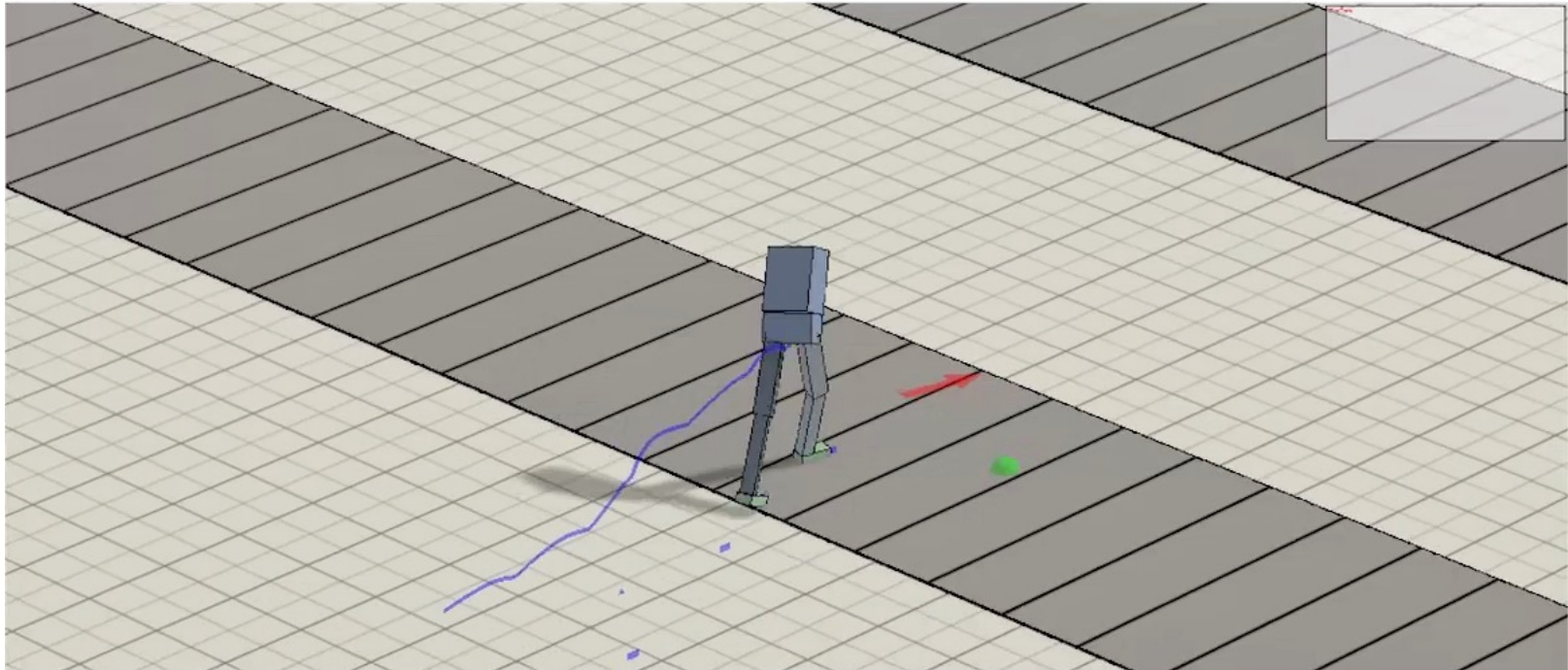
Anymal: Quadrupedal locomotion + getup (ETH Zurich)



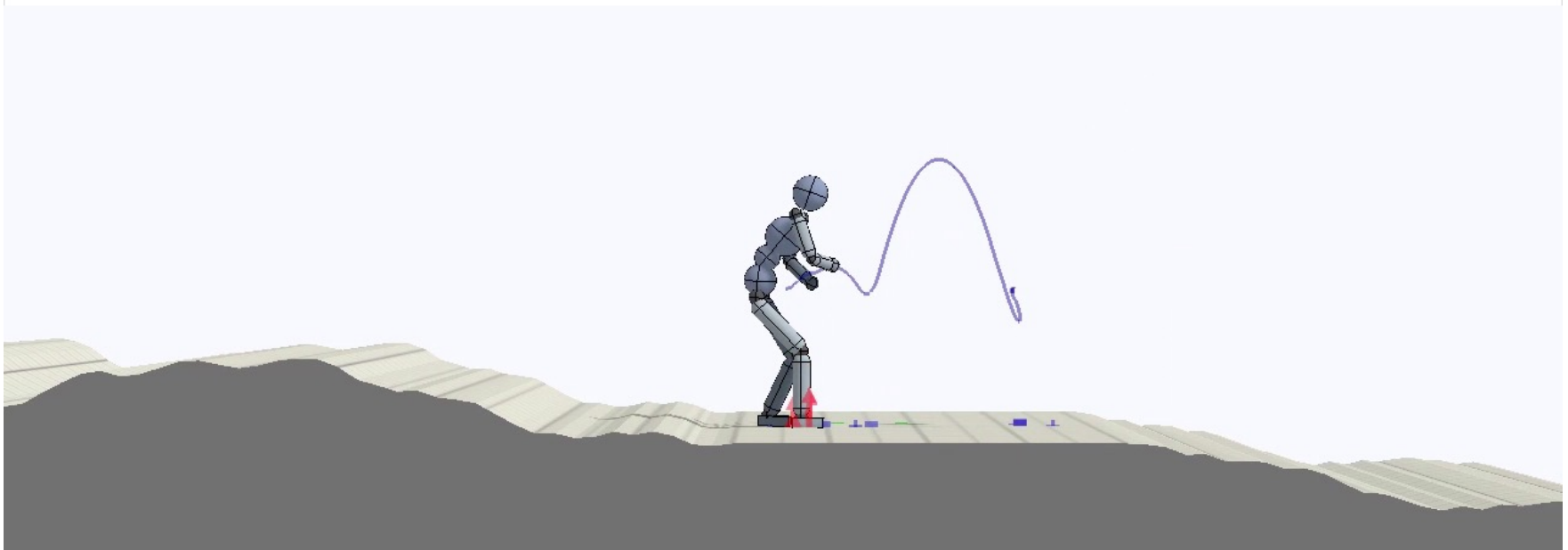
[Science Robotics, 2018]

[DeepLoco: SIGGRAPH 2017]

Walking on Conveyor Belts

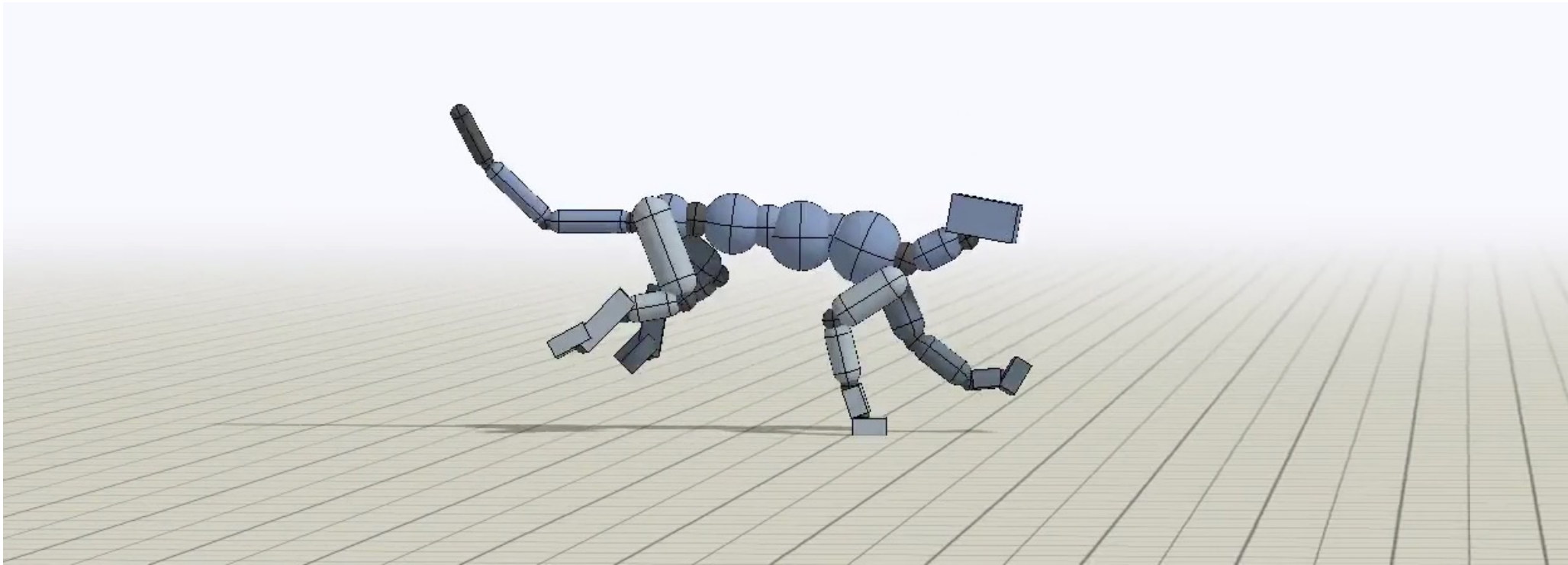


Retargeting



and retarget to different environments.

simulated lion



Physics-based Lion (UC Berkeley, UBC, Ziva Dynamics)



["DeepMimic", SIGGRAPH 2018]

Cassie: Bipedal Locomotion (UBC, U Oregon, Agility Robotics)



Why Deep RL? (continued)

End-to-end learning: performance benefits



Traditional approach



End to end learning

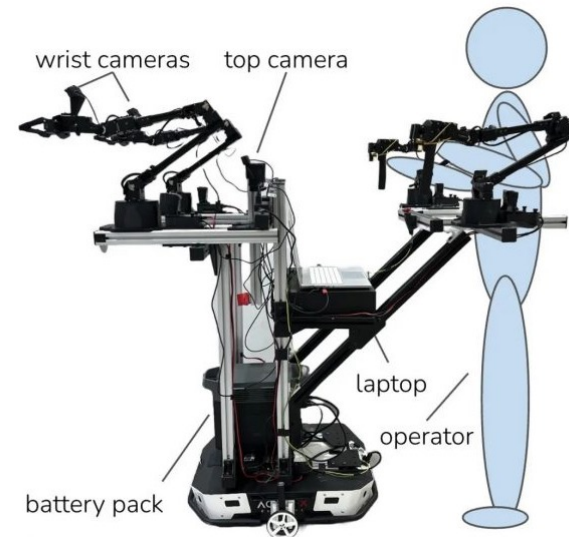


Self-optimized

[Chen and Huang, IV 2017]

“pixels to torques”

Good control from rich sensory streams
is often the limiting factor for robotics



[mobile ALOHA]

[<https://robotik.dfki-bremen.de/en/research/robot-systems/exoskelett-aktiv-ca.html>]

We may better understand how humans and animals learn

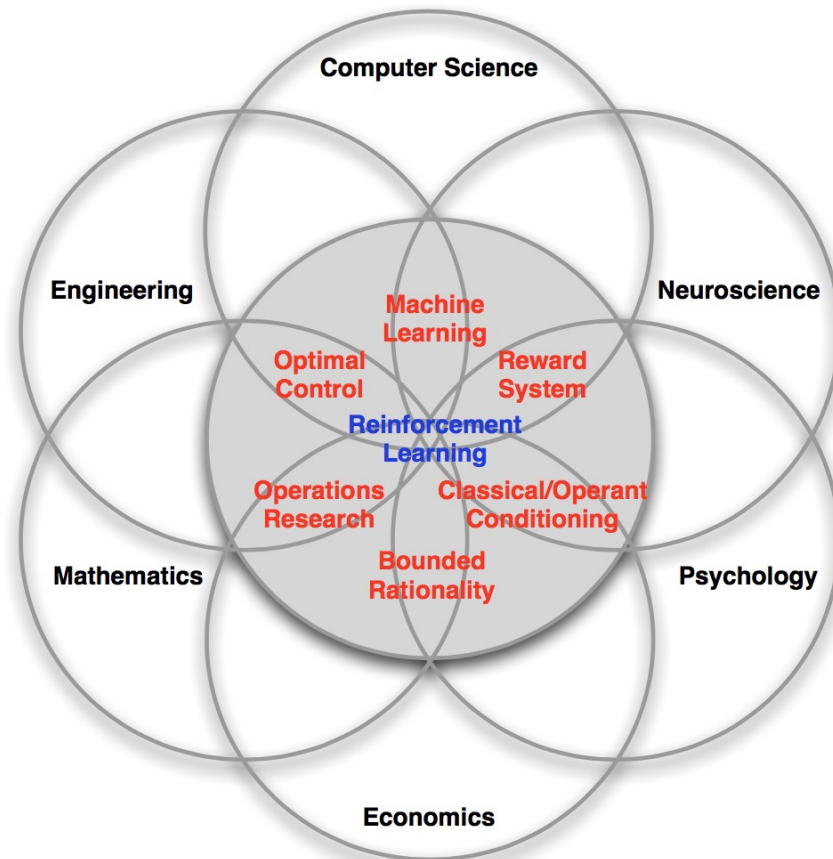


Follow the money

- Alphabet invests ~ \$2B in DeepMind
- Microsoft invests \$1B in OpenAI
- autonomous driving: Mobileye, Wayve, Waymo, Tesla, ...

Some Caveats

RL has a long history (by many names)



[Silver]

Generic Algorithm?

Best results often still have domain knowledge

- domain knowledge
 - defining: state, action, reward
 - hyperparameters
 - example expert data

but:

- no-free lunch: *inductive bias* needed for efficient learning
 - build on foundation models?

RL learning often has poor “sample efficiency”

- often far too slow to learn directly in the real world
 - e.g., thousands of years in simulation (OpenAI hand)
- learning requires good simulation models, so not truly “model free”
- many newborn animals can walk within minutes or hours: giraffe, horse foal, piglets, camels, zebra and more

but:

- compute is cheap
- transfer learning and generalization from related tasks: foundation models
- nature: has done much learning on an evolutionary time scale; this is a form of transfer learning



Alternatives to RL

- AI planning: given a known model of how action impacts world
 - e.g., online search through future states

- Imitation learning: just copy an expert!
 - reduces RL to supervised learning from an expert
 - collect data, learn policy
 - failure modes exist due to cumulative nature of errors
 - combining imitation + RL is a promising direction

$$\{(s_i, a_i)\}$$
$$\downarrow$$
$$a = \pi(s)$$



<https://umi-gripper.github.io/>

RL has limited data to learn from

[Yann LeCun 2016]

Y. LeCun

How Much Information is the Machine Given during Learning?

- ▶ **“Pure” Reinforcement Learning (cherry)**
 - ▶ The machine predicts a scalar reward given once in a while.
 - ▶ **A few bits for some samples**
- ▶ **Supervised Learning (icing)**
 - ▶ The machine predicts a category or a few numbers for each input
 - ▶ Predicting human-supplied data
 - ▶ **10→10,000 bits per sample**
- ▶ **Self-Supervised Learning (cake génoise)**
 - ▶ The machine predicts any part of its input for any observed part.
 - ▶ Predicts future frames in videos
 - ▶ **Millions of bits per sample**



RL can be difficult to apply in practice

Reinforcement Learning for Real Life

ICML 2019 Workshop

June 14, 2019, Long Beach, CA, USA

- Production systems
- Autonomous driving
- Business management
- Chemistry
- Computer Systems
- Energy
- Healthcare
- Robotics/manufacture

“Challenges of Real-World
Reinforcement Learning”
[Dulac-Arnold et al.]

“ ... the research advances in RL are often hard to leverage in real-world systems due to a series of assumptions that are rarely satisfied in practice”

1. Training off-line from the fixed logs of an external behavior policy.
2. Learning on the real system from limited samples.
3. High-dimensional continuous state and action spaces.
4. Safety constraints that should never or at least rarely be violated.
5. Tasks that may be partially observable ...
6. Reward functions that are unspecified, multi-objective, or risk-sensitive.
7. System operators who desire explainable policies and actions.
8. Inference that must happen in real-time at the control frequency of the system.
9. Large and/or unknown delays in the system actuators, sensors, or rewards.

Reproducibility and Brittle Results

Deep Reinforcement Learning that Matters

**Peter Henderson^{1*}, Riashat Islam^{1,2*}, Philip Bachman²
Joelle Pineau¹, Doina Precup¹, David Meger¹**

¹ McGill University, Montreal, Canada

² Microsoft Maluuba, Montreal, Canada

In recent years, significant progress has been made in solving challenging problems across various domains using deep reinforcement learning (RL). Reproducing existing work and accurately judging the improvements offered by novel methods is vital to sustaining this progress. Unfortunately, reproducing results for state-of-the-art deep RL methods is seldom straightforward. In particular, non-determinism in standard benchmark environments, combined with variance intrinsic to the methods, can make reported results tough to interpret. Without significance metrics and tighter standardization of experimental reporting, it is difficult to determine whether improvements over the prior state-of-the-art are meaningful. In this paper, we investigate challenges posed by reproducibility, proper experimental techniques, and reporting procedures. We illustrate the variability in reported metrics and results when comparing against common baselines and suggest guidelines to make future results in deep RL more reproducible. We aim to spur discussion about how to ensure continued progress in the field by minimizing wasted effort stemming from results that are non-reproducible and easily misinterpreted.

[2017]

“Unfortunately, reproducing results for state-of-the-art deep RL methods is seldom straightforward.”

Course Administrivia

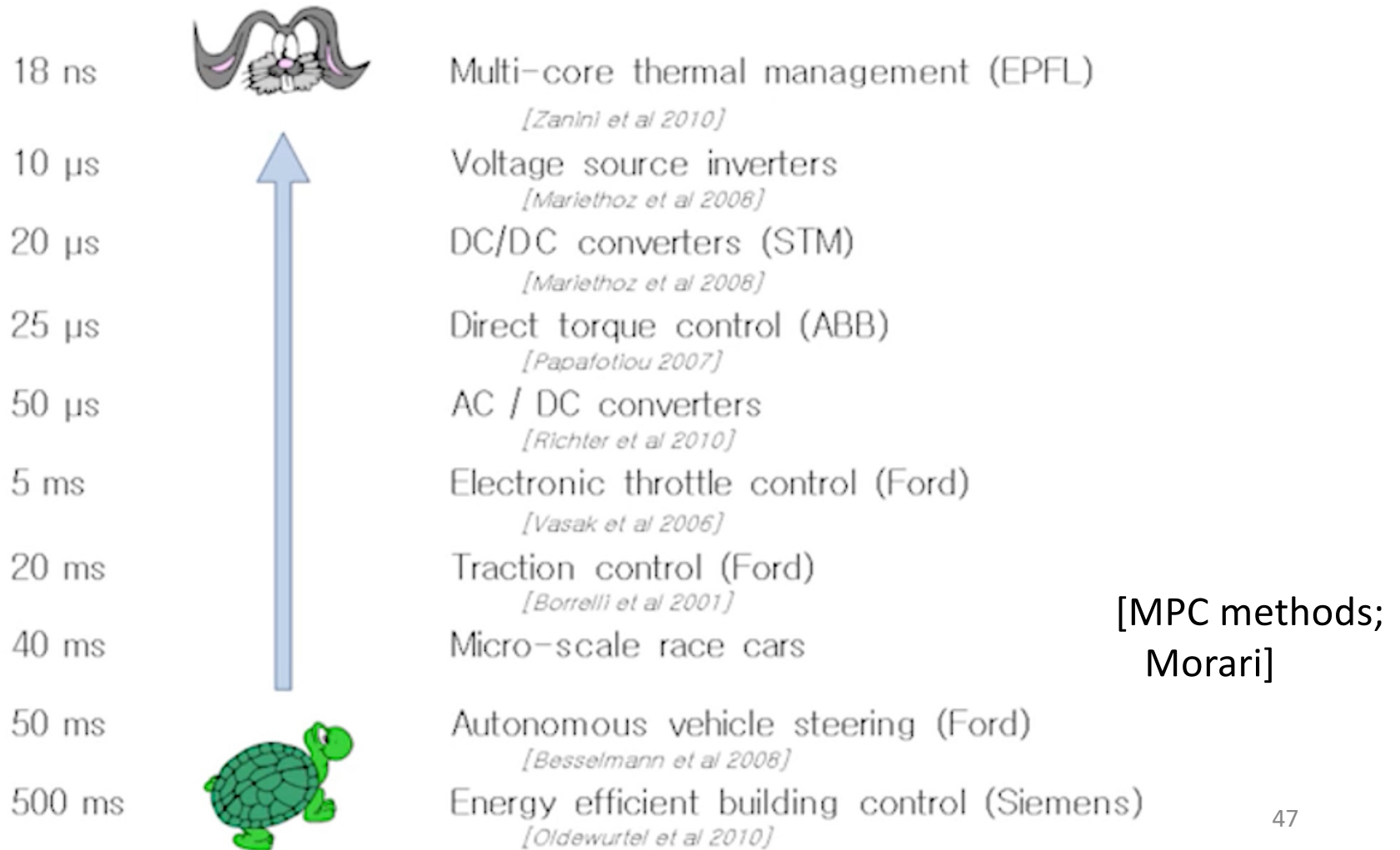
- registration / enrollment
- Piazza
- Assignments
- Project

Icebreaker – get to know other students

RL Basics

CS and Engineering views; Markov Property
(see handwritten notes)

Decision Problems with various time scales



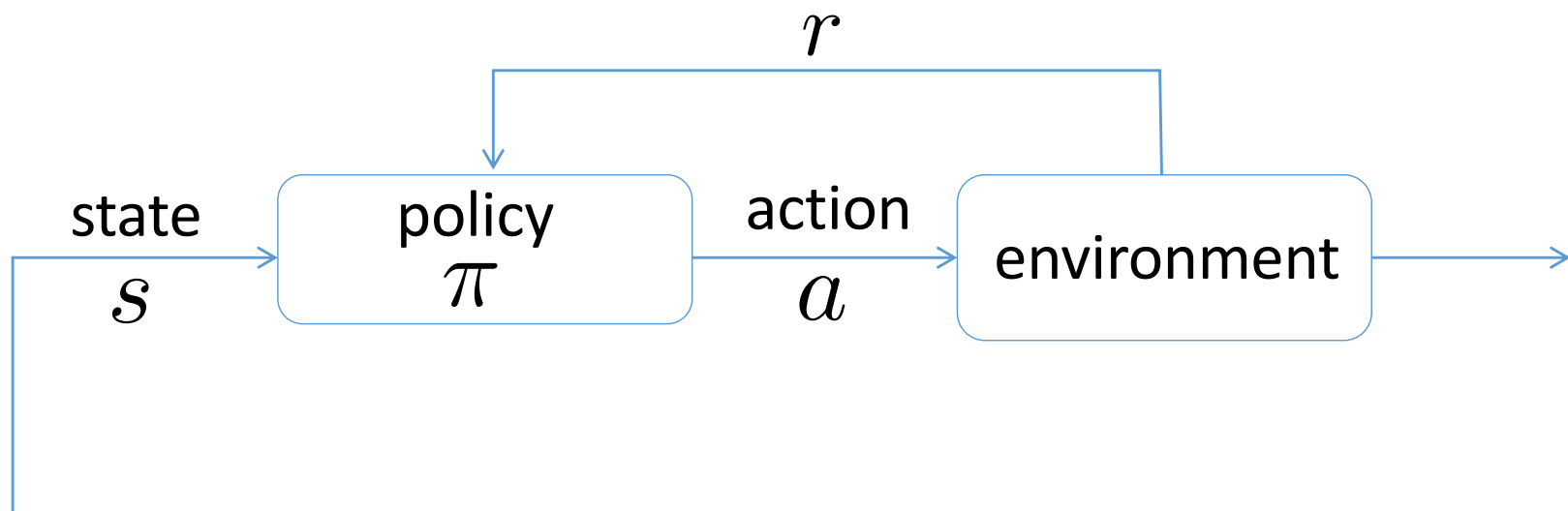
RL Terminology

- states, observations, actions, rewards
- environment / dynamics / state transition probability
- return, expected return
- imitation learning, behaviour cloning, NN distillation
- MDP, POMDP
- episodic task, discounting, stochastic policy

Markov Decision Process (MDP)

(S, A, P, R, γ)

Reinforcement Learning – basics



maximize $G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

“return”, cumulative discounted rewards

A Simple Solution to Learning a Policy: Copy an expert “Imitation Learning”

Observe what an expert does, and try to copy it:

$$\{(s_i, a_i)\}$$



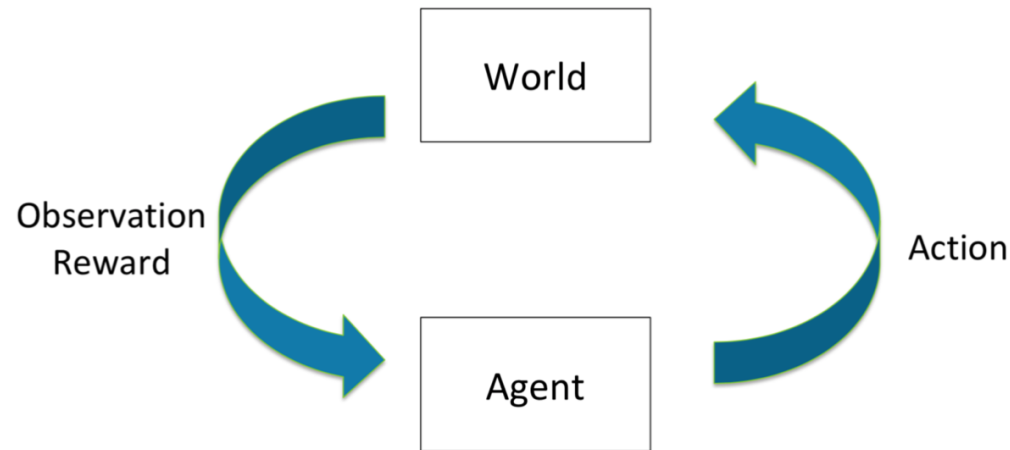
$$a = \pi(s)$$

“behaviour cloning”;

regresses actions onto states
(supervised learning)

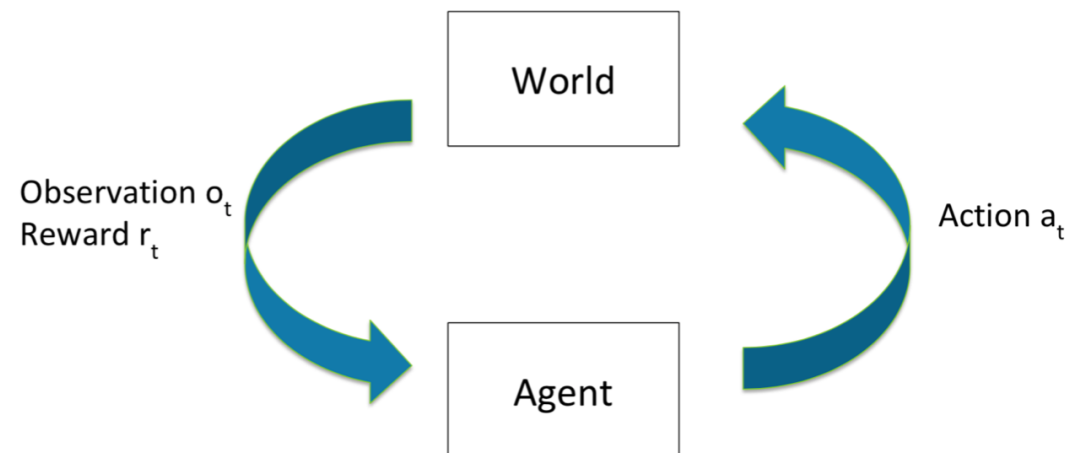
- requires an expert
- learned policy is unaware of the actual task (!)
- suffers from compounding errors over time; can be mitigated by:
 - collecting data during perturbations
 - collecting expert data while following the student policy (“DAGGER” paper)
- transfer between two Deep-NN policies: “policy distillation” “policy cloning”

Sequential Decision Making



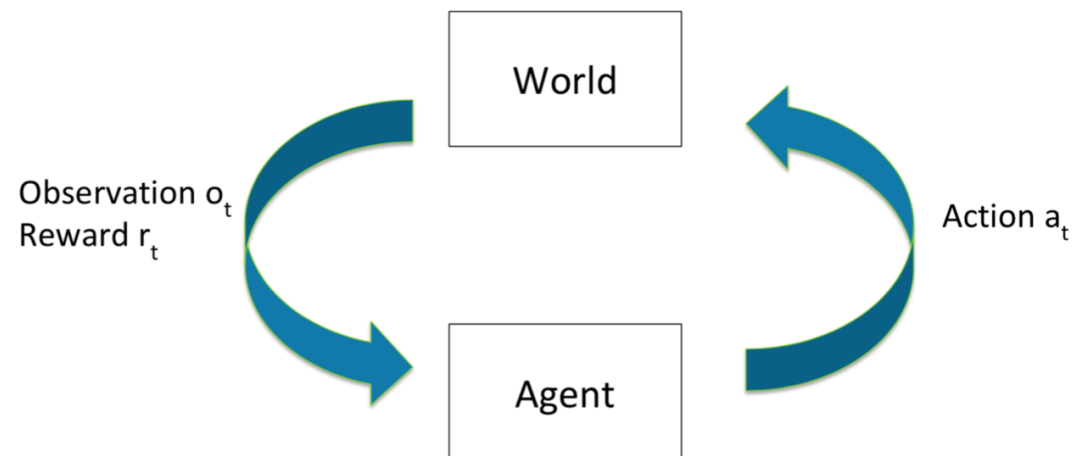
- Goal: Select actions to maximize total expected future reward
- May require balancing immediate & long term rewards
- May require strategic behavior to achieve high rewards

Sequential Decision Process: Agent & the World (Discrete Time)



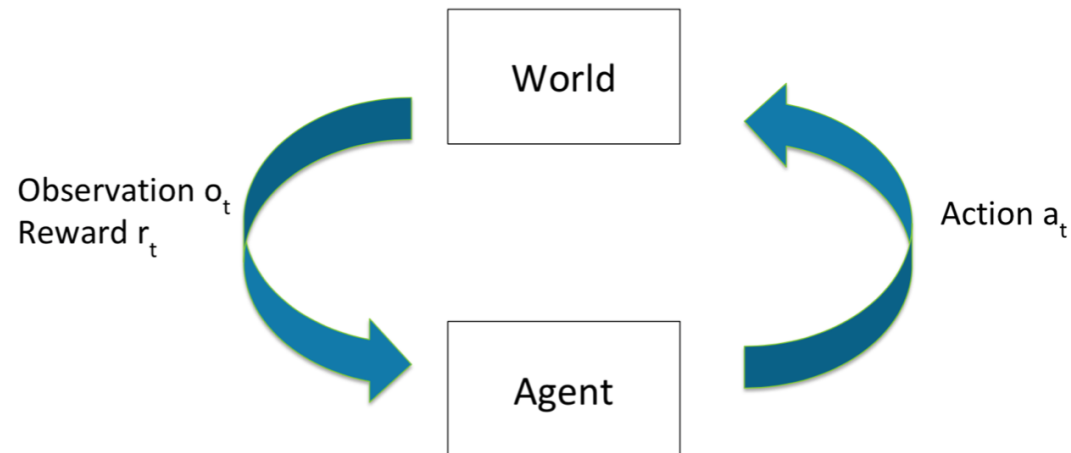
- Each time step t :
 - Agent takes an action a_t
 - World updates given action a_t , emits observation o_t and reward r_t
 - Agent receives observation o_t and reward r_t

History: Sequence of Past Observations, Actions & Rewards



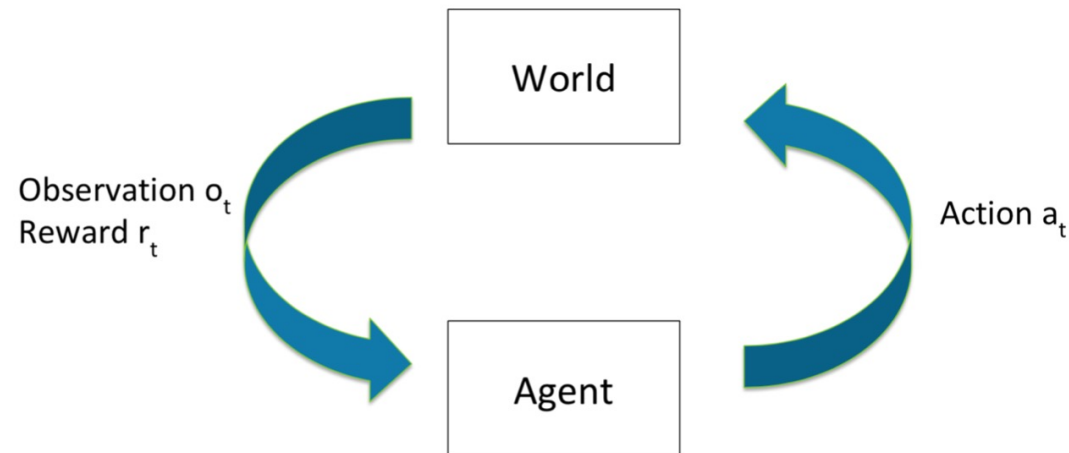
- History $h_t = (a_1, o_1, r_1, \dots, a_t, o_t, r_t)$
- Agent chooses action based on history
- State is information assumed to determine what happens next
 - Function of history: $s_t = (h_t)$

World State



- This is true state of the world used to determine how world generates next observation and reward
- Often hidden or unknown to agent
- Even if known may contain information not needed by agent

Agent State: Agent's Internal Representation



- What the agent / algorithm uses to make decisions about how to act
- Generally a function of the history: $s_t = f(h_t)$
- Could include meta information like state of algorithm (how many computations executed, etc) or decision process (how many decisions left until an episode ends)

Markov Assumption

- Information state: sufficient statistic of history
- State s_t is Markov if and only if:

$$p(s_{t+1}|s_t, a_t) = p(s_{t+1}|h_t, a_t)$$

- Future is independent of past given present

Markov Assumption for Prior Examples

- Information state: sufficient statistic of history
- State s_t is Markov if and only if:

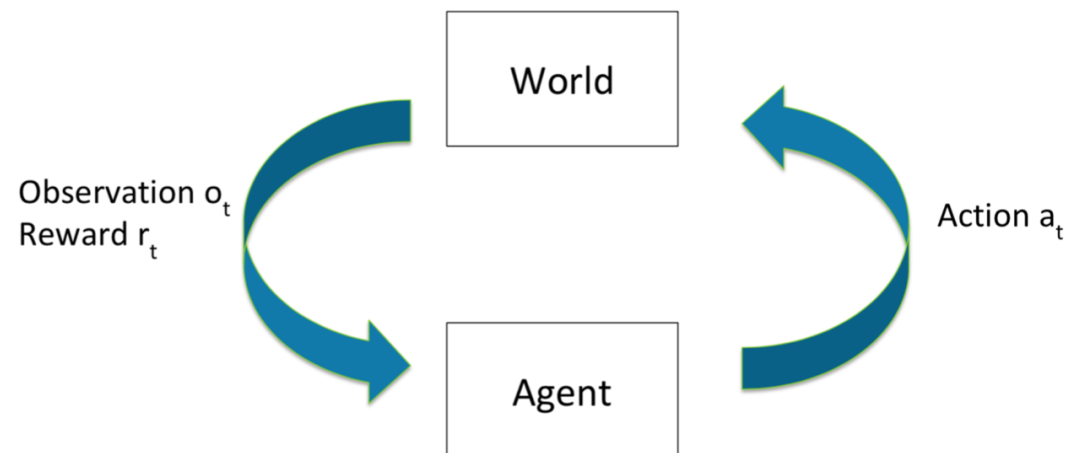
$$p(s_{t+1}|s_t, a_t) = p(s_{t+1}|h_t, a_t)$$

- Future is independent of past given present
- Hypertension control: let state be current blood pressure, and action be whether to take medication or not. Is this system Markov?
- Website shopping: state is current product viewed by customer, and action is what other product to recommend. Is this system Markov?

Why is Markov Assumption Popular?

- Can always be satisfied
 - Setting state as history always Markov: $s_t = h_t$
- In practice often assume most recent observation is sufficient statistic of history: $s_t = o_t$
- State representation has big implications for:
 - Computational complexity
 - Data required
 - Resulting performance

Full Observability / Markov Decision Process (MDP)



- Environment and world state $s_t = o_t$

Markov Decision Process (MDP)

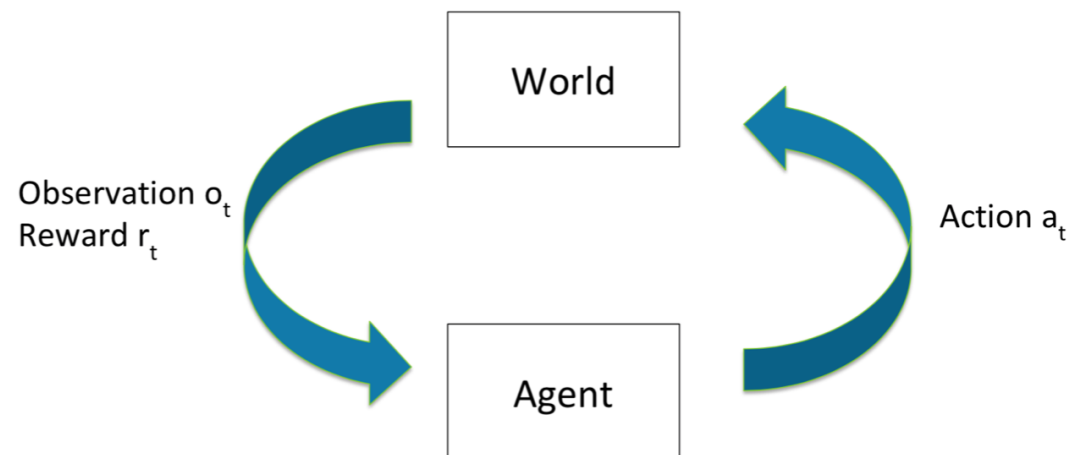
- Markov Decision Process is Markov Reward Process + actions
- Definition of MDP
 - S is a (finite) set of Markov states $s \in S$
 - A is a (finite) set of actions $a \in A$
 - P is dynamics/transition model for **each action**, that specifies $P(s_{t+1} = s' | s_t = s, a_t = a)$
 - R is a reward function¹

$$R(s_t = s, a_t = a) = \mathbb{E}[r_t | s_t = s, a_t = a]$$

- Discount factor $\gamma \in [0, 1]$
- MDP is a tuple: (S, A, P, R, γ)

¹Reward is sometimes defined as a function of the current state, or as a function of the (state, action, next state) tuple. Most frequently in this class, we will assume reward is a function of state and action

Partial Observability / Partially Observable Markov Decision Process (POMDP)



- Agent state is not the same as the world state
- Agent constructs its own state, e.g.
 - Use history $s_t = h_t$, or beliefs of world state, or RNN, ...

Common RL Assumptions

- *episodic* tasks
 - repeated interactions with the world e.g., games, trials
 - begins from a standard starting state (or distribution of states)
 - ends when reaching terminal state, or a fixed time T
- *discounting*: γ
 - avoid infinite rewards when $T = \infty$ (arguably a hack: could optimize mean reward)
 - uncertainty about the future:
“probability that the game will be allowed to continue after the current move”
 - useful regularizer for temporal difference learning
 - “effective horizon”: $H = \frac{1}{1 - \gamma}$
- *stochastic policies*: action probabilities $\pi(a|s)$ vs $a = \pi(s)$
 - “smooths” the learning, e.g., probability of playing a card, steering left, etc
 - provides exploratory actions (for some algorithms)
 - often switch to using a deterministic policy once learning is complete

More on discounting

I always thought in reinforcement learning we try to conflate too many ideas into one number, γ , the discount factor. It serves as :

- (1) a technical tool to avoid computing with infinities*
- (2) a way to deal with a finite but uncertain lifetime, and*
- (3) a way to summarize our uncertainty about the future.*

In economics, there's also

- (4) a prediction about inflation rates or equivalently the net present value of a future financial instrument.*

I think we'd be better off if we tried to model each of these things separately, instead of saying we'll have one hyperparameter and try to learn the best value for it.

-- Peter Norvig

RL (MDP) problem attributes

- state: {discrete, continuous, hybrid}
- action: {discrete, continuous, hybrid}
- initial state: {deterministic, stochastic}
- policy: {deterministic, stochastic}
- dynamics: {deterministic, stochastic}
- dynamics: {known, unknown (black-box), modeled via learning}
- state: {known, unknown (only have [stochastic] "observations")}
- time: {discrete, continuous}
- rewards: {deterministic, stochastic}
- task: {episodic, continuous}

Research ideas exist in all directions!

- how can we have humans provide rewards? (RL from human feedback)
- given a system and its behavior, can we infer the rewards? (inverse RL)
- can we learn without a fixed task? (continual/lifelong learning)
- how do we learn from similar tasks? (transfer learning; universal policies)
- should we learn easy tasks, then difficult tasks? (learning curricula)
- how can we leverage expert (or non-expert) data? (imitation methods; offline RL)
- can we learn how to learn? (meta-learning)
- how can we leverage known-or-modeled dynamics? (model-based RL)
- how can we meaningfully evaluate RL results? (“Deep RL that matters”)
- what is the best way to debug RL algorithms?